# Things I've been building, lately

I've been having a lot of fun exploring projects lately. Below are just a handful of a projects I've been messing with. Some are utilities. Some are learning projects. All of them have been incredibly fun and helped along with AI to various degrees.

## // beagle

Dynamic, mostly-functional, compiles straight to machine code

No VM, no bytecode, no JIT warm-up. By far my longest running most complete project. All compilation and code generation are dependency free custom backends for x86 and arm. The goal is to be a language with fantastic live coding support allowing for humans or ai to build full applications without ever restarting a process.

● beagle examples/echo_server.bg

```
use beagle.socket as socket

fn main() {
    let listener = socket/listen("0.0.0.0", 8080)
    println("Echo server listening on port 8080")

    socket/on-connection(listener, fn(conn) {
        loop {
            let data = socket/read(conn, 4096) // fully async, no function coloring
            if data == "" || data == null {
                socket/close(conn)
                break(null)
            } else {
                socket/write(conn, data)
            }
        }
    })
}
```

# FEATURE RICH LANGUAGE

01 **Algebraic effects**
Built-in algebraic effects make it easy to control and test io

02 **Live coding**
Redefine functions, structs, and enums while your code is running

03 **Async without function coloring**
Delimited continuations give us the power to have transparent asynchrony

04 **Compiled**
no VM, no bytecode — direct machine code

## // keep-running

Dtach, but with a friendlier CLI and a replay buffer

Run a long process, detach, let it keep going, reattach later from anywhere. Small utility, but I've found it incredibly useful

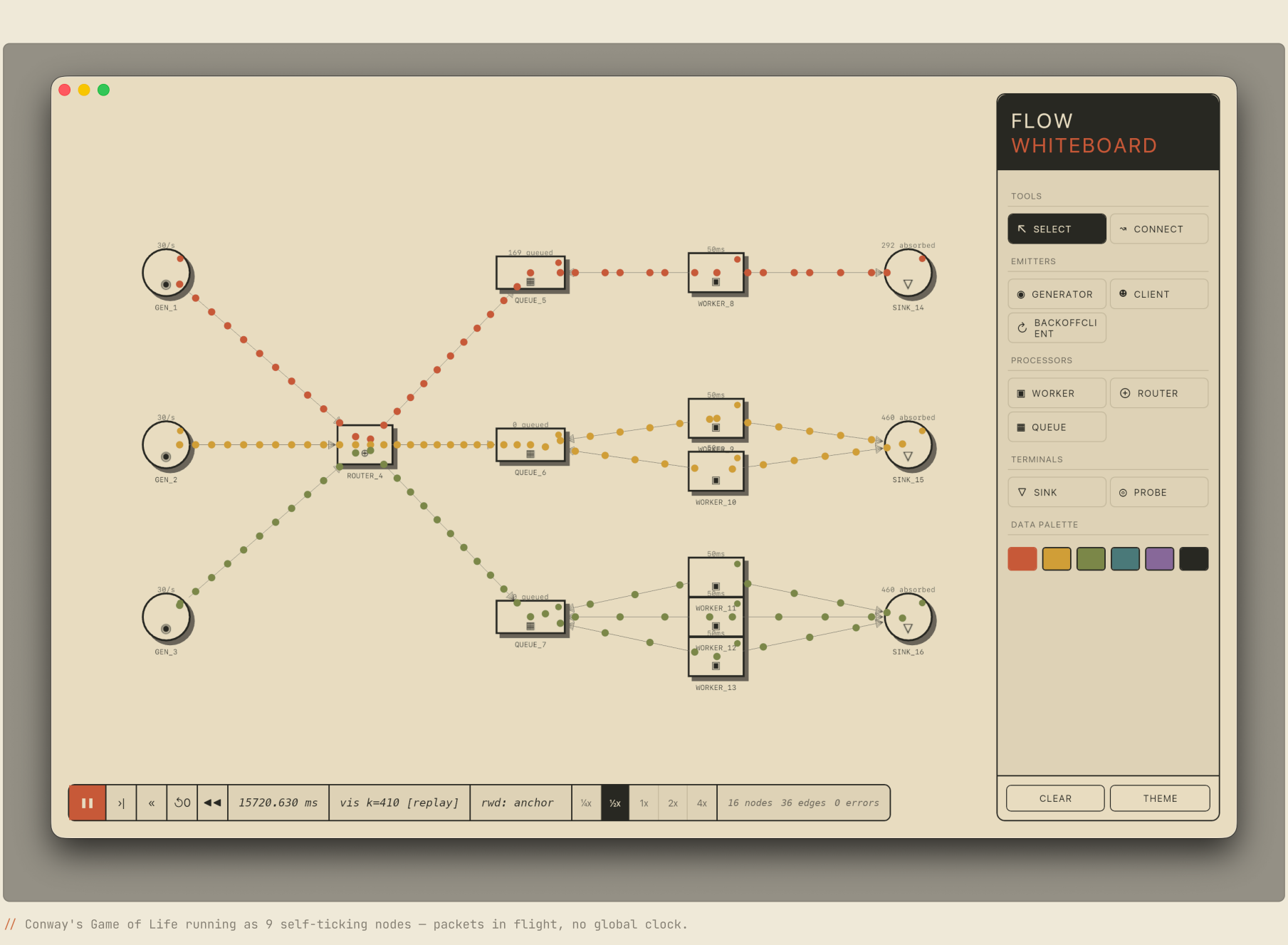
~/keep-running

```
$  keep-running run -n build -- cargo build --release
   → session 'build' started · pid 84219
   → detach with ctrl-\ ctrl-\

$  keep-running ls
   NAME    PID     STATUS     UPTIME    BUF
   build   84219   running    14m       482 KB
   tests   84402   running     3m        91 KB

$  keep-running attach build
   → replaying last 482 KB from buffer...
       Compiling porter v0.4.1 (/Users/_/code/porter)
       Compiling tokio v1.36.0
   → live
$
```

## // bevy-whiteboard

The diagram on the whiteboard, but it actually runs

I've always wanted to be able to draw on a whiteboard and then truly simulate the system. This proof-of-concept does exactly that. Under the hood is a simulation driven by a rules-based language. Enabling running simulations faster than real time or rewinding through time. It is quickly approach something powerful and flexible enough to model real systems.

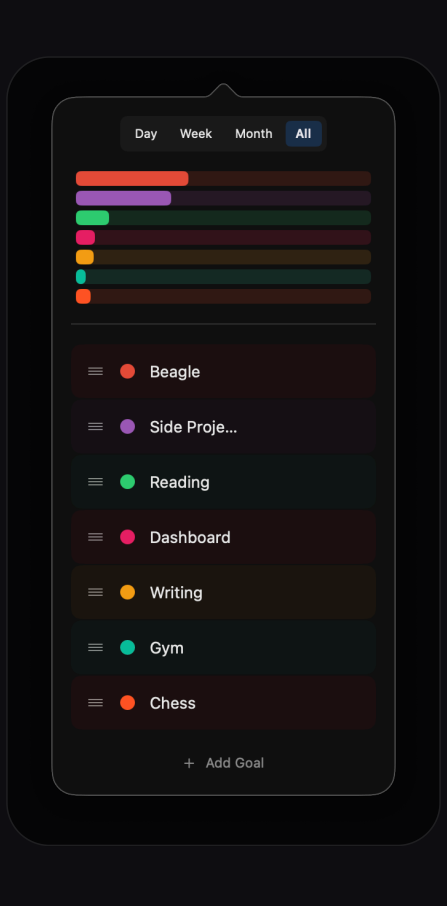


// Conway's Game of Life running as 9 self-ticking nodes — packets in flight, no global clock.

## // Ease

A macOS menu-bar habit tracker that I actually use

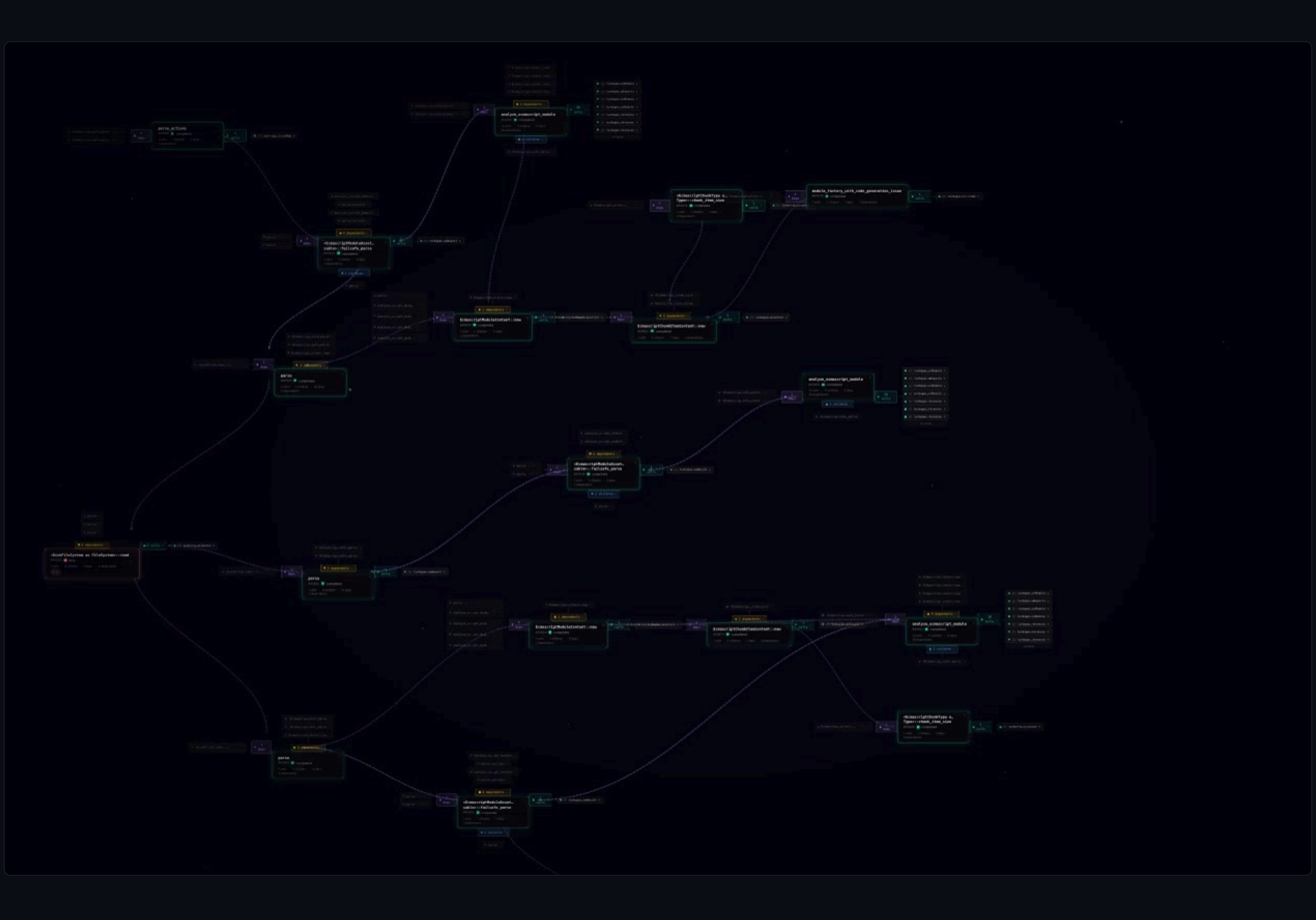
Ease is an incredibly simple, but handy progress tracker I've made for myself. Rather than track exact time I work on projects, it lets me fuzzily measure the amount of work I've put in



## // turbopack-visualizer

Untapped Way to Learn a Codebase: Build a Visualizer

I have aphantasia. From my very first job I realized if I want to fully understand a codebase, I needed to build something that visualizes how it works and let's me interact with it. This is now how I learn new codebases. I wrote a blog post where I walk through how I do this using turbopack as my case study. Below is the video of the quick Vc visualizer and stepping debug that was the end result of the endeavor.



## // simd-lang

A language where every value is a SIMD vector

I've always wanted to explore what a language that took simd as the primitive would look like. This was a fun exploration. I was able to with 53 lines of a dsl (simplified below by cutting out unicode handling) get something that does the stage1 parsing of json about .75x the speed of simd json. The whole language compiles using mlir taking full advantage of its vector dialect. In many ways this is a learning project for me to start better understanding simd approaches.

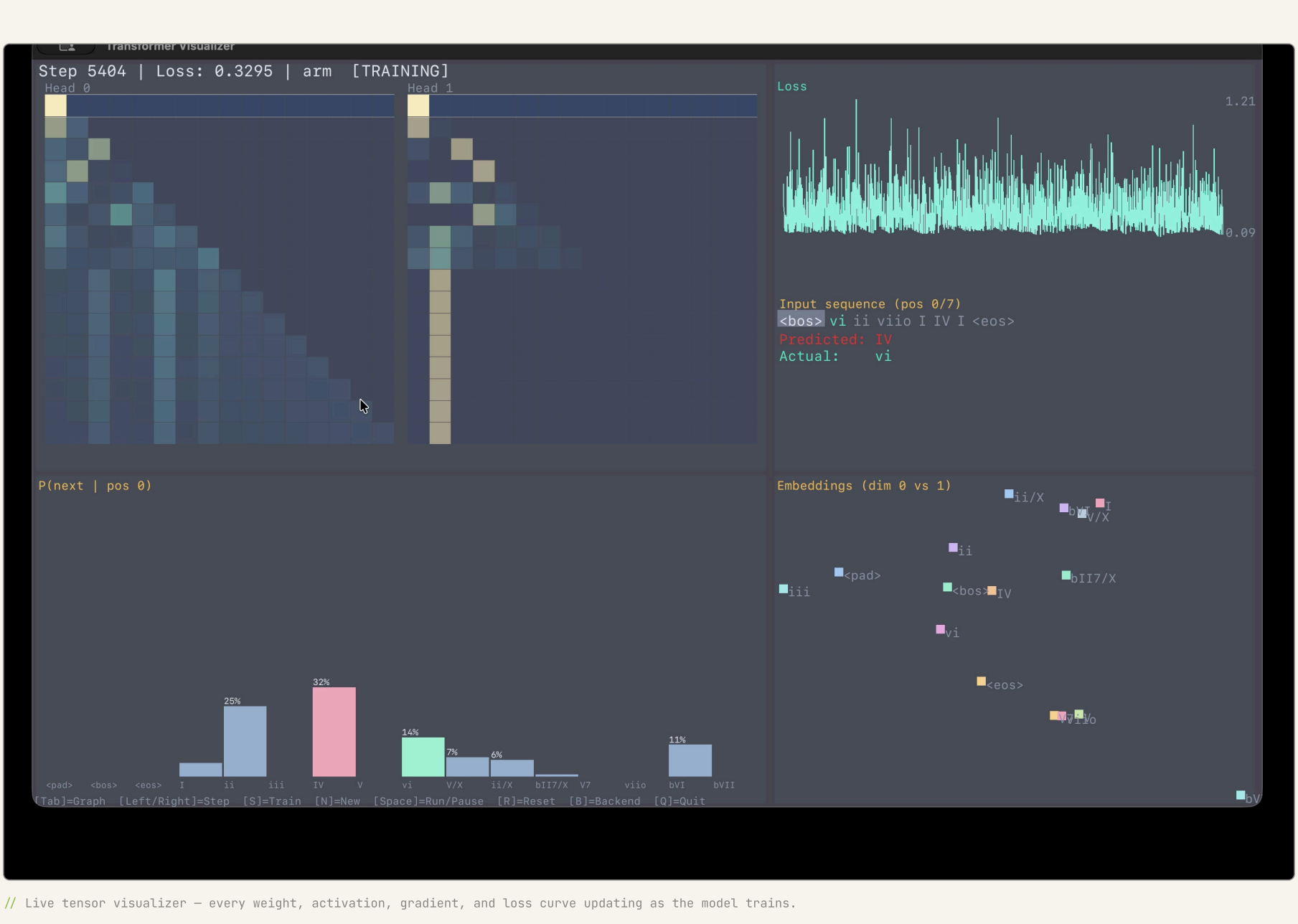
● simd examples/json_stage1.simd

```
fn json_stage1(input: ptr[u8], positions: ptr[i32]) -> i32[1] {
    stream chunk: u8[64] over input carry (prev_in_string: u64[1] = 0,
                                           pos: i32[1] = 0) {
        quote_vec      = chunk == '"'
        bs_vec         = chunk == '\\'
        real_quote_vec = quote_vec & ~lane_shr(bs_vec, 1)

        structural_vec = (chunk == '{') | (chunk == '}') | (chunk == '[')
                       | (chunk == ']') | (chunk == ':') | (chunk == ',')

        all_ones: u64[1]     = ~0
        quote_bits           = to_bitmask(real_quote_vec)
        in_string_raw        = clmul(quote_bits, all_ones)
        in_string_with_carry = in_string_raw ^ prev_in_string

        real_structural_bits = to_bitmask(structural_vec) & ~in_string_with_carry
        real_structural_vec  = from_bitmask(real_structural_bits, 64)

        indices = iota(64) + chunk_offset
        compressstore(positions, pos, indices, real_structural_vec)

        carry prev_in_string = (in_string_with_carry >> 63) * all_ones
        carry pos            = pos + popcount(real_structural_vec)
    }
    return pos
}
```

## // tensor-lang

From-scratch tensor compiler — one DSL, four backends, real models

I've become increasingly interested in how llm compilers do their optimization. But also in deeply understanding the transformer architecture. This is an on going project where I have a simple dsl that can express simple models like gpt2. I think take this and compile it into arm machine code, wasm (scalar), wasm (simd), and wgpu. As part of this I tried a tiny model to learn some jazz chord progressions and made a UI where you can watch it train live and inspect every tensor. All of this is powered by my compilers and dsl.



Step 5404 | Loss: 0.3295 | arm [TRAINING]

Input sequence (pos 0/7)
<bos> vi ii viio I IV I <eos>
Predicted: IV
Actual: vi

[Tab]=Graph [Left/Right]=Step [S]=Train [N]=New [Space]=Run/Pause [R]=Reset [B]=Backend [Q]=Quit

// Live tensor visualizer — every weight, activation, gradient, and loss curve updating as the model trains.

● tensor examples/gpt2.tl — the attention block

```
// Attention block
let ln1_out  = layernorm(x, ln1_g, ln1_b)
let qkv      = linear(ln1_out, attn_qkv_w, attn_qkv_b)
let qkv_r    = reshape(qkv, [B, T, 3, n_head, head_size])
let q        = reshape(shrink(qkv_r, [[0, B], [0, T], [0, 1], [0, n_head], [0, head_size]]), [B, T, n_head, head_size])
let k        = reshape(shrink(qkv_r, [[0, B], [0, T], [1, 2], [0, n_head], [0, head_size]]), [B, T, n_head, head_size])
let v        = reshape(shrink(qkv_r, [[0, B], [0, T], [2, 3], [0, n_head], [0, head_size]]), [B, T, n_head, head_size])
let q_h      = permute(q, [0, 2, 1, 3])
let k_h      = permute(k, [0, 2, 1, 3])
let v_h      = permute(v, [0, 2, 1, 3])
let kt       = permute(k_h, [0, 1, 3, 2])
let scores   = mul(matmul(q_h, kt), inv_head_size)
let masked   = add(scores, mask_full)
let attn     = softmax_attn(masked)
let attn_out = matmul(attn, v_h)
let merged   = reshape(permute(attn_out, [0, 2, 1, 3]), [B, T, n_embd])
let attn_res = linear(merged, attn_proj_w, attn_proj_b)
let x        = add(x, attn_res)
```

| PASS | TIME | SPEEDUP |
|---|---|---|
| baseline (old ARM backend) | 3740 ms | 1.00x |
| + nested loops | 3595 ms | 1.04x |
| + k-invariant hoisting | 3201 ms | 1.17x |
| + matmul unfusion + MR=8 kernel | 1475 ms | 2.54x |
| + pointer incrementing | 1434 ms | 2.61x |
| **+ all combined** | **198 ms** | **18.90x** |
| llm.c -O3 (reference) | 84 ms | — |

// GPT-2 124M · 12 layers · T=16 · single-threaded · Apple Silicon

## // datalog-db

A Datomic-shape database with a real surface syntax

I've grown to love datalog. It is a fascinating way to look at databases and truly gives some amazing expressiveness. This is only a POC right now, but the goal is the build a proper datalog data base on top of a key value store. But I don't want users to need to think about the exact details of the underlying store and instead work in a much higher level fashion. To that end I've added something I always want in a db, algebraic data types.

● datalog repl session

```
define User {
    name:  string required,
    age:   i64,
    email: string unique indexed,
}

define enum Status {
    Active,
    Suspended { reason: string },
}

assert User { name: "Alice",   age: 30, email: "alice@example.com" }
assert User { name: "Bob",     age: 25, email: "bob@example.com"   }
assert User { name: "Charlie", age: 35, email: "charlie@example.com" }

find ?name, ?age
  where ?u: User { name: ?name, age: > 27 }
  as_of_time "2026-04-30T00:00:00Z"
```